

CSS Notes

Cascading Style Sheets — from basics to animations

Basics of CSS

What is CSS?

CSS stands for **Cascading Style Sheet**. It is a styling language, not a programming language, used to describe the *style* of a document. HTML provides structure/layout, CSS provides style, and JS provides logic.

Basic Syntax

```
h1 {
    color: red;
}
```

Selector → h1 **Property** → color **Value** → red

Note: The semicolon marks the end of a property. It's good practice even though the last property in a block won't error out without it.

Including Style — 3 Ways

1. Inline

Vijay Rawat

2. Internal (<style> tag)

[illegible]

3. External Stylesheet (best practice)

Writing CSS in a separate .css file and linking it to the HTML file with a <link> tag.

Note: Priority: inline style overrides internal (<style> tag) style, which overrides an external stylesheet.

Color Property

```
color: red;
color: pink;
color: blue;
color: green;
```

Background Color Property

```
background-color: red;
background-color: pink;
```

```
background-color: blue;
background-color: green;
```

Color Systems

RGB

```
color: rgb(255, 0, 0);
color: rgb(0, 255, 0);
```

Hex (Hexadecimal)

```
color: #ff0000;
color: #00ff00;
```

Note: You don't need to pick colors from memory — use an online color picker tool.

Selectors

- **Universal Selector** — `* { }`
- **Element Selector** — `h1 { }`
- **Id Selector** — `#myId { }`
- **Class Selector** — `.myClass { }`

Practice Set 1

Q1. Create a simple div with id "box". Add some text content inside the div. Set its background color to blue.

Q2. Create 3 headings h1, h2 & h3. Give them all a class "heading" & set color of "heading" to red.

Q3. Create a button and set its background color to: green (using css stylesheet) / blue (using `<style>` tag) / pink (using inline style).

Text Properties

text-align

```
text-align : left / right / center
```

Note: Alignment is relative to the parent element, not the page. CSS3 also introduced 'start' and 'end' for language support (e.g. Arabic).

text-decoration

```
text-decoration : underline / overline / line-through
```

Note: Can also take style (wavy/dotted), color (red), or 'none' — commonly used to remove underline from hyperlinks.

font-weight

```
font-weight : normal / bold / bolder / lighter
font-weight : 100-900
```

Values from 100 to 900 represent how dark/light the text appears.

font-family

```
font-family : arial
font-family : arial, roboto
```

Note: Multiple font families act as a fallback mechanism — if the first isn't available, the browser tries the next.

Generic vs Specific Font Families

- **Generic font families:** defined by shared attributes — serif, sans-serif, monospace, cursive, fantasy.
- **Specific font families:** actual named fonts — Arial, Times New Roman, Tahoma.
- **Serif:** Times New Roman, Garamond, Palatino, Georgia (used in print).
- **Sans-Serif:** Arial, Helvetica, Verdana, Trebuchet MS (default in digital).
- **Cursive:** Comic Sans MS, handwritten look.
- **Fantasy:** stylized, decorative fonts.
- **Monospace:** Courier New, Consolas — all characters equal width.

Units in CSS — Absolute

```
96px = 1 inch  
font-size: 2px;
```

Note: cm, mm, inch etc. also exist, but pixels (px) is the most commonly used absolute unit.

line-height

```
line-height : 2px  
line-height : 3  
line-height : normal
```

text-transform

```
text-transform : uppercase / lowercase / capitalize / none
```

Practice Set 2

Q1. Create a heading centred on the page with all its text capitalized by default.

Q2. Set the font family of all the content in the document to "Times New Roman".

Q3. Create one div inside another div. Set id & text "outer" for the first, "inner" for the second. Set outer div text size to 25px & inner div text size to 10px.

Box Model in CSS

The Box Model

Every element is a box made up of four parts, from inside out: **Content** → **Padding** → **Border** → **Margin**.

Height & Width

```
div {  
  height: 50px;  
  width: 50px;  
}
```

By default, these set the **content area's** height/width of the element.

Border

```
border-width : 2px;  
border-style : solid / dotted / dashed  
border-color : black;
```

Shorthand

```
border : 2px solid black;
```

Border Radius

```
border-radius : 10px;  
border-radius : 50%;
```

Used to round the corners of an element's outer border edge.

Padding

Space between content and border: padding-left, padding-right, padding-top, padding-bottom.

Shorthand

```
padding: 50px;  
padding: 1px 2px 3px 4px;
```

Note: Order for the 4-value shorthand: top | right | bottom | left (clockwise).

Margin

Space outside the border, between this element and neighboring elements: margin-left, margin-right, margin-top, margin-bottom.

Shorthand

```
margin: 50px;  
margin: 1px 2px 3px 4px;
```

Note: Same clockwise order applies: top | right | bottom | left.

Practice Set 3

Q1. Create a div with height & width of 100px. Set its background color to green & the border radius to 50%.

Q2. Create an amazon.in style navbar: black background, 60px height, orange (#f08804) search button, black (#0f1111) text color, 25px text size, links via anchor tags, ~200px gap between links.

Display Property

```
display: inline / block / inline-block / none
```

- **inline** — takes only the space required by the element (no margin/padding effect on layout).
- **block** — takes the full available width.
- **inline-block** — like inline, but margin & padding can be applied.
- **none** — removes the element from document flow entirely.

Visibility

```
visibility: hidden;
```

Note: When visibility is 'hidden', space for the element is still reserved. When display is 'none', no space is reserved at all.

Alpha Channel — RGBA

```
color: rgba(255, 0, 0, 0.5);  
color: rgba(255, 0, 0, 1);
```

The 4th value is **opacity**, ranging from 0 (transparent) to 1 (fully opaque).

Practice Set 4

Q1. Create a webpage layout with a header, footer & a content area containing 3 divs. Set height & width of divs to 100px (add the previous navbar in the header).

Q2. Add borders to all the divs.

Q3. Add a different background color to each div with an opacity of 0.5.

Q4. Give the content area an appropriate height.

Position

Units in CSS — Relative

- **%** — relative to an element's parent object.
- **em** — relative to font size (see below).
- **rem** — relative to the root element's font size.
- **vh / vw** — relative to 1% of viewport height / width.

Percentage (%)

```
width : 33% ;  
margin-left : 50% ;
```

Note: Sizing relations are the most common use, but percentage can occasionally relate to other properties too.

em

Relative to the **font size of the parent** for typographical properties (like font-size), and relative to the **element's own font size** for other properties (like width). For padding & margin, em is relative to that same element's font size.

rem (Root Em)

Always relative to the **font size of the root element** (<html>), regardless of nesting — avoids the compounding effect of em.

Position Property

```
position : static / relative / absolute / fixed
```

- **static** — default position. top/right/bottom/left/z-index have no effect.
- **relative** — positioned relative to itself; top/right/bottom/left/z-index now work.
- **absolute** — positioned relative to its closest *positioned* ancestor; removed from normal flow.
- **fixed** — positioned relative to the browser viewport; removed from flow, stays put on scroll.
- **sticky** — positioned based on the user's scroll position.

z-index

```
z-index : auto (0)
z-index : 1 / 2 / ...
z-index : -1 / -2 / ...
```

Decides the **stack level** of elements. Overlapping elements with a larger z-index cover those with a smaller one.

Background Image

```
background-image : url("image.jpeg");
```

Background Size

```
background-size : cover / contain / auto
```

- **cover** — fills the box completely, no empty space (may crop image).
- **contain** — fits the image fully visible within the box.
- **auto** — original image size.

Practice Set 5

Qs. Create the given layout with `<p>lorem*5</p>`, `<div>Love Nature</div>`, `<p>lorem*5</p>`:

- Give the div a height, width & some background image.
- Use the appropriate position property to place the div at the right end of the page (should not move on scroll).
- Use z-index to place the div on top of the page.

Flexbox

Flexbox — Flexible Box Layout

A **one-dimensional** layout method for arranging items in rows or columns.

The Flex Model

Set `display: flex;` on the container first. Key terms: **flex container**, **flex item**, **main axis**, **cross axis**.

Flexbox Direction

```
flex-direction : row;           /* default */
flex-direction : row-reverse;
flex-direction : column;
flex-direction : column-reverse;
```

Sets how flex items are placed in the flex container — along which axis and direction.

Flex Properties — For the Flex Container

- **justify-content** — alignment along the *main axis*: flex-start / flex-end / center / space-evenly / space-between / space-around.
- **flex-wrap** — nowrap / wrap / wrap-reverse.
- **align-items** — alignment along the *cross axis*.
- **align-content** — alignment of space between & around content along the cross-axis (multi-line).

Flex Properties — For a Flex Item

- **align-self** — alignment of an individual item along the cross axis (overrides align-items).
- **flex-grow** — how much an item grows relative to siblings if extra space is available (0, 1, 2... ; 0 = doesn't grow).
- **flex-shrink** — how much an item shrinks relative to siblings if space is insufficient.

Practice Set 6

Qs. Create a navbar with 4 options as anchor tags inside list items. Use flexbox to place them spaced equally in a single line.

Qs. Use flexbox to center one div inside another div.

Qs. Which has higher priority — align-items or align-self? (align-self wins for that individual item.)

Media Queries

Help create a **responsive** website that adapts to different screen sizes and orientations (phones, tablets, laptops, big screens).

```
@media (width : 600px) {
  div {
    background-color : red;
  }
}

@media (min-width : 600px) {
  div {
    background-color : red;
  }
}
```

Combining Conditions

```
@media (min-width : 200px) and (min-width : 300px) {
  div {
    background-color : red;
  }
}
```

Practice Set 7

Qs. Add media queries so a div's color changes to:

- green for viewport width < 300px
- pink for width between 300px & 400px
- red for width between 400px & 600px
- blue for width above 600px

***Note:** Good to know conceptually, even if used less often in day-to-day styling.*

Transitions

Transitions define a smooth change between two states of an element (e.g. on hover).

```
transition-property : property you want to transition (font-size, width, etc.)
transition-duration : 2s / 4ms ...
transition-timing-function : ease-in / ease-out / linear / steps ...
transition-delay : 2s / 4ms ...
```

Shorthand

property name | duration | timing-function | delay

```
transition: font-size 2s ease-in-out 0.2s;
```

CSS Transform

Used to apply 2D & 3D transformations to an element.

rotate

```
transform: rotate(45deg);
```

scale

```
transform: scale(2);
transform: scale(0.5);
transform: scale(1, 2);
transform: scaleX(0.5);
transform: scaleY(0.5);
```

translate

```
transform: translate(20px);
transform: translate(20px, 50px);
transform: translateX(20px);
transform: translateY(20px);
```

***Note:** Distances can also use %, em, rem, and negative values are valid.*

skew

```
transform: skew(30deg);
```

Animation

Used to animate CSS elements over defined keyframes.

```
@keyframes myName {
  from { font-size : 20px; }
  to   { font-size : 40px; }
}

@keyframes myName {
  0%   { font-size : 20px; }
  50%  { font-size : 30px; }
  100% { font-size : 40px; }
```



```
}
```

Animation Properties

- animation-name
- animation-duration
- animation-timing-function
- animation-delay
- animation-iteration-count
- animation-direction

Shorthand

```
animation : myName 2s linear 3s infinite normal;
```

Practice Set 8

Qs. Create a simple loader using CSS:

Step 1 — Create a div with a circular shape & a thick border from one end (top/bottom/left/right).

Step 2 — Create an animation which transforms it from 0deg to 360deg to make it spin.

Step 3 — Add the animation property to the loader with infinite duration.